Októberi beszámoló

Dinamikus szobanövény-monitorozó rendszer hardveres és AI technológiával

Az októberi céljaim a szakdolgozatom keretében készülő projektemre vonatkozóan a következők voltak:

- Az IoT prototípus fizikai összeállítása
- Az ESP32 mikrokontroller programozása
- Szenzoradatok beolvasása
- HTTP kérésekkel történő adat továbbítás

A projekt következő szakaszát az alkatrészek kicsomagolásával és ellenőrzésével kezdtem. Ismétlésnek itt van a megvásárolt alkatrészek, szenzorok pontos listája:

- ESP-WROOM-32-ESP32S (mikrokontroller)
- DHT22-M (hőmérséklet, páratartalom)
- GY-30 - BH1750FVI (környezeti fény)
- SOILCAP-V20 (talajnedvesség)
- BB-102 (630/200) (próbapanel)
- RC-10-20/MF (kábelek)
- RC-10-20/FF (kábelek)
- RC-10-20/MM (kábelek)
- CCP-MUSB2-AMBM-1M (micro usb kábel)

Első ránézésre minden rendben volt, viszont sajnos egy gyártói döntés miatt a fényérzékelő modul 5 pin-es csatlakozója nem került ráforrasztásra a modulra. Ezért a forrasztást saját magamnak kellett elvégezni, amely a folyamat végső tesztelését meghosszabbította. Szerencsére a fényérzékelő nélkül is el tudtam kezdeni a munkát.

Rögtön a legelső lépésnél egy kellemetlen hibába ütköztem. A próbapanel, ami összeköti a mikrokontrollert a szenzorokkal 5-5 oszlopnyi területtel rendelkezik, viszont az ESP32 modulom fizikális kialakítása miatt 9 pin szélességben helyezkedik el a próbapanelen. Ez azért probléma, mert így nem lehet szépen rátenni a panel közepére, mert csak az egyik oldalon marad szabad oszlop, viszont a talajnedvesség mérő szenzort a másik kettővel ellentétben a bal oldalra kell bekötni. Ez azért van, mert analóg jelet küld a szenzor, viszont az ESP32 Wi-Fi használata közben nem tud analóg jelet fogadni, ezért ezt a modult a bal oldalon található [VP] lábba kell bedugni, amely képes az analóg jelet digitálisra váltani Wi-Fi használat közben is. A megoldás tehát az lett, hogy az ESP32-őt a próbapanel aljára, csak a jobb oldali pin-eket dugtam bele, rögtön a legelső (a) oszlopba. Így a bal oldal szabadon “lelóg” a levegőbe, így hozzáférhető lesz a [VP] láb.

A szenzorok közül mindhárom 3.3V-os tápellátással üzemel, ezért így az ESP32 jobb oldalon található 3.3V lábát tudjuk használni a próbapanelen mindenhol. Ezt rákötöttem a jobb oldali [+] sínre, míg a GND lábát a [-] sínre a földelés miatt. Így tehát az áramellátás és a földelés könnyen szétoszthatóa szenzorok között. Ehhez MM(Male-to-Male) kábeleket használtam

Először a DHT22-M került a panelre. Ezt a legtetején helyeztem el, függőlegesen az (e) oszlopra. A tápellátást a VCC, a földelést a GND kimenet kezeli (ez azonos lesz mindegyik szenzornál, tehát ismételni nem fogom), így ezeket bedugtam a [+] és a [-] sínekbe. Nagyon fontos részlet volt, hogy az MM kábeleket szigorúan abba a sorba kellett dugni, amelyik sorban a szenzor adott kimenete is van, ugyanez érvényes az ESP32 esetében is. A DHT22 DATA kimenetét a [D4] lábhoz kötöttem be.

A SOLICAP-V20 volt a következő. Ennek van egy saját 3 pines csatlakozója egy hosszabb kábellel, mivel ezt majd valamelyik növény földjébe kell dugni. Egyelőre még nem hosszabbítottam meg, csak 2 másik MM kábellel továbbítottam a tápellátást, földelést, illetve az AOUT kimenetet. Most jön vissza az, amiről fentebb írtam, ez analóg jelet küld, ezért a próbapanel helyett a levegőben lógó [VP] lábhoz kötöttem be egy MF(Male-to-Female) kábellel.

Egy gyorsabb forrasztással használhatóvá vár végül a G-30 - BH1750FVI szenzor is, amely a környezeti fény mérését végzi. Ezt a DHT22-M alatt helyeztem el 10 sorral lentebb, a (b) oszlopban, hogy az (a) oszlop hozzáférhető legyen. Az SCL kimenetet a [D22]-es lábhoz, az SDA kimenetet a [D21]-es lábhoz kötöttem be. Ennél a lépésnél sajnos elfogyott az MM kábelem, így kettő MF kábelt feláldozva forrasztás segítségével csináltam egy saját gyártású MM kábelt.

Ezzel tehát elkészült a fizikai prototípus, következett az eszköz programozása. Fejlesztői környezetnek a VS Code PlatformIO bővítményét használtam, mert ezzel gyorsan, korszerű eszközökkel tudtam elvégezni a tesztelést és a megvalósítást. Létrehoztam egy projektet, mely során grafikus felületen ki tudtam választani az ESP32-őt mely a rendszerben “DOIT ESP32 DevKit V1” néven volt megtalálható, segítségemre volt az is, hogy ez az elnevezés rá is volt nyomtatva a mikrokontroller hátlapjára. A projekt automatikusan létrejött egy “platformio.ini” fájllal, amely tartalmazza a felhasznált duependenciákat és konfigurációt. Rövid kutatással megtaláltam mindegyik szenzorhoz a megfelelő könyvtárakat, illetve a “bblanchon/ArduinoJson” könyvtárat, amely az adatokat biztonságosan json formátummá alakítja.

A “main.cpp” fájlban történik a fő logika. A kód elején minden szükséges csomagot importáltam. A következő lépésben deklaráltam a szenzorokat a megfelelő pin-eken, a megfelelő osztályokkal. A DHT-22 és a BH1750 saját változót kap, a SOILCAP V-30 által küldött adatokat rögtön az analogRead() metódussal olvassuk ki az ESP32 VP bemenetén. Az egész kód egy folyamatos ciklus, amely 15 percenként (tesztelés miatt ez jelenleg 10 másodperc) - egy időzítő segítségével - beolvassa a szenzorok méréseit, ezeket becsomagolja egy JSON csomagba, és egy HTTP POST kéréssel továbbítja a kód elején deklarált szerver címre a megadott api kulccsal. Nagyon fontos, hogy a küldés csak aktív Wi-Fi kapcsolattal kezd bele a küldésbe, ezzel kerülve a felesleges forgalmat. Most ezek a kérések tesztelés miatt a “http://httpbin.org/post" internetes eszközhöz érkeznek be, amellyel tudtam tesztelni a program stabil és helyes működését.

Az adatok a következő mértékegységekben érkeznek be a szenzorokról:

> "Hő: 23°C | Pára: 59% | Talaj: 2300 | Fény: 200 lx”

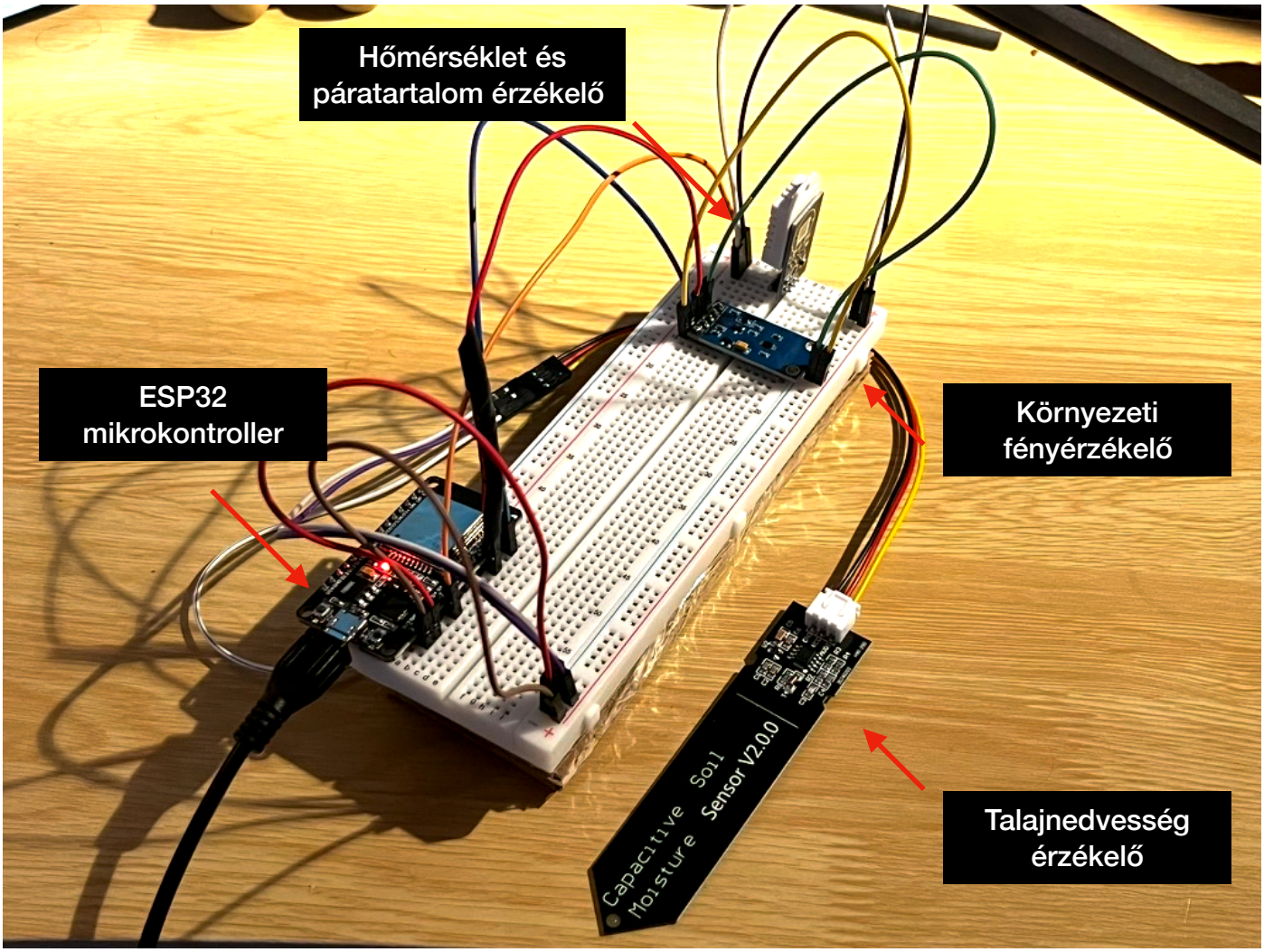
Első ránézésre a talajnedvesség értéke egy értelmezhetetlen számérték. A szenzor gyárilag 0-4096 intervallumban tud értéket kimutatni, amely szerint a 0 a legnedvesebb, míg a 4096 a legszárazabb érték. Ennek a kalibrációját a funkcionális tesztelés során fogom elvégezni, a szerveroldali kódban fogom kezelni az átváltást értelmezhető értéktartományra. Ugyanez vonatkozik a többi értékre is.

Azért döntöttem a 15 perces mérés mellett, mert ez egy elég nagy ugrás két időpont között ahhoz, hogy egy pillanatnyi változás ne zavarja meg az értékeket, pl. valaki beáll a fényérzékelő elé, pillanatai lámpa felkapcsolás, stb. A másik indok pedig az erőforrásaim ingyenesen tartása. A Vercel ingyenes limitjei miatt a legbiztonságosabb út, hogy 15 percenként küldjem a szerverre az adatokat, ami naponta 96, havonta kb 2880, tehát bőven van helye még más api hívásoknak is havi szinten.

A fizikai prototípusról lentebb elhelyeztem pár képet, természetesen ezt a projekt közben tervezem szebb, prezentálhatóbb megjelenésűre átvarázsolni.

A projekt fejlesztését tovább folytatom, a fejleményekről továbbra is részletesen beszámolok a következő hónapokban.

Köszönöm a figyelmet!


Hőmérséklet és páratartalom érzékelő
ESP32 mikrokontroller
Környezeti fényérzékelő
Talajnedvesség érzékelő
Capacitive Soil Moisture Sensor V2.0.0

